

Maxwell

Automatic Differentiation by Sparse Graph Vertex Elimination

Implementation Report and User Guide

Software and Tools for Scientific Computing

9 September 2026

Abstract

Maxwell is a C++ operator-overloading library for automatic differentiation. It records a program's arithmetic as a directed acyclic graph and computes the Jacobian by *vertex elimination* rather than by a forward or reverse sweep, which lets the elimination direction be chosen per problem and makes the cost of differentiation a property of the graph rather than of the traversal. Its distinguishing feature is *checkpointing under a memory budget*: a tape too large to hold is broken into partitions, and the recorded section is replayed once per partition, so a deep computation can be differentiated in bounded memory. The partitioning is bit-exact — the same Jacobian is produced at a 1 GB budget and at a 1 kB budget, to the last bit, with twelve pinned witnesses asserting it.

This report documents the implementation and the reasoning behind it. Part I is a maintainer's reference: the graph representation, the elimination core, the checkpoint protocol, the memory accounting, the tape object, the RAI surface, the two storage arenas, and what is and is not possible concurrently. Design choices are justified with the measurements that produced them, including several that were measured *wrong* before they were measured right — a memory model that under-reported by a factor of 2.44, an arena whose first two parameter choices each made things worse, and a scope-restoration bug that only a constructed test could expose. Part II is a user guide: how to drive the library, what the checkpoint contract requires of your code, how to choose a budget, and six worked use cases taken from the shipped examples.

All measurements were taken on GCC 13.3.0, -O2, x86-64, and are reproducible from the tree with `make test` and `make sanitize`.

Contents

I	Design and Implementation	4
1	Introduction	5
1.1	What the library does	5
1.2	What makes it different	5
1.3	A note on the listings	6
1.4	Scope of this report	6
1.5	The source tree	6
2	Recording: the graph	7
2.1	The active scalar	7
2.2	How an operation is recorded	8
2.3	Reachability	9
2.4	The graph	9
3	Elimination	10
3.1	The operation	10
3.2	Order matters, and adjacency order does not	11
3.3	Direction	11
3.4	Harvest	12
4	Checkpointing	13
4.1	The idea	13
4.2	Partitions and the owner index	13
4.3	The replay loop	14
4.4	The contract, which is the sharpest edge in the library	15
4.5	The property that makes it trustworthy	15
4.6	What replay costs	15
5	The memory budget	17
5.1	What a graph costs	17
5.2	SVEGP-24: the accounting never counted adjacency	17
5.3	The two errors were cancelling	17
5.4	The drift alarm	18
5.5	SVEGP-25: a budget that was not a budget	18
6	The tape object	19
6.1	SVEGP-26: two global states that had to agree	19
6.2	What that bought, precisely	20
6.3	SVEGP-27: the compiler found what had to move, not what had to be guarded	20
6.4	SVEGP-27: ownership belongs in a destructor	20

7	The RAII surface	22
7.1	Why	22
7.2	A wrapper, not a second implementation	22
7.3	Scopes	23
7.4	SVEGP-29: a bug only a constructed test could find	23
7.5	Mixing the two surfaces	24
7.6	Why the free functions stayed	24
8	Storage: the two arenas	25
8.1	Where the memory went	25
8.2	Step 1: the edge arena	25
8.3	Step 2: sorted adjacency	26
8.4	Step 3: the adjacency arena	26
9	Concurrency	29
9.1	What is shared	29
9.2	What works	29
9.3	What does not work, and why	30
9.4	A caveat on diagnostics	30
10	Verification	31
10.1	Structural values and accounting values	31
10.2	Controls	31
10.3	Sanitizers	32
10.4	What the suite asserts	32
II	Using the Library	33
11	Getting started	34
11.1	Building	34
11.2	Including it	34
11.3	The shape of a program	34
12	The tape surface	36
12.1	Construction	36
12.2	Independents	36
12.3	Running the section	36
12.4	Dependents, harvest, and the rest	36
12.5	The Jacobian	37
13	The checkpoint contract	38
14	Choosing a budget	39
14.1	What the number means	39
14.2	How to pick one	39
14.3	What chunking costs	39
14.4	Elimination direction	39

15 Use cases	41
15.1 A gradient: many inputs, one output	41
15.2 A full Jacobian: n inputs, n outputs	41
15.3 A two-dimensional grid	42
15.4 An adjoint: data assimilation	42
15.5 Verifying a derivative	43
15.6 Independent tapes in parallel	43
16 Pitfalls	45
16.1 <code>active</code> is not a value type	45
16.2 Do not hold actives across tapes	45
16.3 Do not mix the two surfaces	45
16.4 The budget is bytes	46
16.5 Check <code>empty()</code>	46
A Defect log	47
B Measurement summary	48
B.1 Graph representation	48
B.2 Cost of chunking	48
B.3 Parallel independent tapes, 2 cores	48
B.4 Suite status	49
C Known limitations	50

Part I

Design and Implementation

Chapter 1

Introduction

1.1 What the library does

Maxwell differentiates a C++ program by overloading the arithmetic operators on a scalar type, `maxwell::active`. Where a program computes with `double`, it computes with `active` instead; each elementary operation appends a vertex and one or two edges to a graph whose edge weights are the local partial derivatives. Once the program has run, the Jacobian of the declared dependents with respect to the declared independents is obtained by eliminating every intermediate vertex from that graph.

The public protocol, in its oldest form, is seven ordered steps:

```
initialize(n,m,budget);
independent(x[i]) ...
while( checkpoint(x,y) ){ try{ f(); } catch(BreakException const&){} }
dependent(y[k]) ...
harvest(m,n,A);           // double**, caller-owned
free_jacobian(m,A);
finalize();
```

Part II describes the modern surface, in which a `Tape` object replaces `initialize/finalize`, owns the loop and its `try/catch`, and returns the Jacobian as a value. The free functions above remain, are still public, and are what the object surface is built on — Chapter 7 explains why they were kept rather than retired.

1.2 What makes it different

Two things.

Vertex elimination, not a sweep. Forward-mode and reverse-mode AD are the two extreme orderings of a more general operation: eliminating vertices from the linearised computational graph. Maxwell implements the general operation, so the direction is a setting (`set_elim_mode`) rather than a different code path. On a deep, narrow tape the difference is large and measurable: `examples/heat Assim` reports 6 835 multiply-adds in reverse against 112 102 forward on the same graph, a ratio of 16.4×, for an identical Jacobian.

Checkpointing under a byte budget. This is the feature the rest of the design serves. The library is told how many bytes of graph it may hold. It runs the user's section once to *profile* it, discovers how many partitions the budget implies, and then replays the section once per partition, recording only one partition at a time and eliminating each into the accumulating result. Deep time-stepping problems that would otherwise exhaust memory become tractable,

and — crucially — the answer does not depend on the budget. The regression suite asserts this at seven budgets from 200 000 down to 1 000 bytes with *zero* deviation, not a tolerance.

1.3 A note on the listings

Code in this report is taken from the tree, not paraphrased. Listings are excerpted — elisions are marked ... — and occasionally carry a short explanatory comment that is not in the source. Where a listing shows a defect it is labelled as such and the fixed form is given alongside. Every snippet was checked against the file it came from before this document was compiled.

1.4 Scope of this report

Chapter 2 covers the recording machinery and the graph representation. Chapter 3 covers elimination. Chapter 4 covers checkpointing and the contract it imposes on user code. Chapter 5 covers the memory accounting that the budget is spent against. Chapter 6 covers the tape object that replaced a singleton. Chapter 7 covers the RAII surface. Chapter 8 covers the two storage arenas and the measurements that shaped them. Chapter 9 covers concurrency. Chapter 10 covers how the library is verified, which is a design choice in its own right.

1.5 The source tree

<code>maxwell.hpp</code>	the public surface; includes nothing internal
<code>inc/Active.hpp</code>	the <code>active</code> scalar and its operators
<code>inc/API.hpp</code>	the free-function protocol
<code>inc/Tape.hpp</code>	<code>maxwell::Tape</code> and <code>maxwell::Jacobian</code> (public)
<code>inc/TapeState.hpp</code>	<code>internals::TapeState</code> — one tape's whole state
<code>inc/Process.hpp</code>	the recorder and eliminator
<code>inc/Vertex.hpp</code>	<code>Vertex</code> , <code>Adjacency</code> , <code>AdjArena</code>
<code>inc/Edge.hpp</code>	<code>Edge</code> , <code>EdgeArena</code>
<code>inc/Typedefs.hpp</code>	<code>largeint</code> , <code>elim_t</code>
<code>src/*.cpp</code>	six translation units
<code>examples/</code>	nine programs; <code>make test</code> runs all of them

`maxwell.hpp` deliberately does not include `Vertex.hpp`, `Edge.hpp`, `Process.hpp` or `TapeState.hpp`. That is not tidiness: it is what allowed the graph representation to be replaced twice (Chapter 8) without a single caller changing, and it is why the public `Tape` holds one forward-declared pointer rather than its state by value.

Chapter 2

Recording: the graph

2.1 The active scalar

`active` is a 48-byte value carrying its numeric value and its position in the graph:

```
class active
{
public:
    mutable bool    reachable;    // does this value depend on an independent?
    mutable largeint idx;         // vertex index, 0 if passive
    mutable largeint owner_idx;   // which partition recorded it
    mutable largeint old_idx;     // saved across a replay
    mutable double   val;         // the value itself
    mutable internals::Vertex * vtx; // cached vertex, or NULL
    ...
};
```

Every member is `mutable` so that the operators can take `const active&` and still update bookkeeping. This is defensible for `idx` and `vtx`, which are cache-like state on a logically constant object; it is less defensible for `val` and `reachable`, which are the object's actual value, and it means `const active&` promises less than it appears to. It is recorded here as a known wart rather than defended.

active is not a value type. Copying one *records*. The copy constructor is:

```
active::active( const active & x ):
reachable(x.reachable), idx(0), owner_idx(0), old_idx(0), val(x.val), vtx(NULL)
{
    unary_op( x , 1.0 , *this , false );    // an identity vertex and edge
}
```

ADOL-C and CoDiPack do the same, and it is the only way an overloading library can track a copy. But nothing in the type says so, and the standard library copies things constantly. Measured on `y = x0*x1 + x0`:

what the program does	tape size
the straight-line expression	480 bytes
... with one <code>active</code> passed <i>by value</i>	600 bytes
... with 8 <code>push_back</code> into <code>std::vector<active></code>	2 640 bytes

Eight `push_backs` into a growing vector cause $1 + 2 + 4 + 8 = 15$ reallocation copies, and every copy records an identity vertex and edge: exactly the 15×144 bytes measured. The derivative

is still correct; the tape is five and a half times larger than it should be, and nothing warns. Section 16 tells users how to avoid it; Section 12.2 describes the library-side mitigation, which is that the tape now hands out its own independents from storage sized exactly once.

2.2 How an operation is recorded

Each overloaded operator computes the value and the local partials, then calls one of five recording entry points. Multiplication is representative:

```
active maxwell::operator*( const active & x1 , const active & x2 )
{
    active x3;
    x3.val = x1.val*x2.val;
    x3.reachable = x1.reachable || x2.reachable;
    binary_op( x1 , x2.val , x2 , x1.val , x3 );    // dy/dx1 = x2, dy/dx2 = x1
    return x3;
}
```

`binary_op` reaches the current tape and forwards to `Process`, whose behaviour differs sharply between the two passes:

```
void Process::binary_op( const active & x1 , double dy_dx1 ,
                        const active & x2 , double dy_dx2 , const active & x3 )
{
    if(profiling){
        if(x3.reachable){
            x3.idx = next_vertex_idx++;
            intmed_count++;
            if(&x1==&x2){ edge_count++; }
            else{
                if(x1.reachable) edge_count++;
                if(x2.reachable) edge_count++;
            }
        }
    }else{
        Vertex * v1=NULL , * v2=NULL , * v3=NULL;    // SVEGP-20
        ...
        if(x3.reachable){
            x3.idx = next_vertex_idx++;
            v3 = vertex_on_lhs(x3);
            if(&x1==&x2){ add_edge(v1,v3,dy_dx1+dy_dx2); edge_count++; }
            else{
                if(x1.reachable){ add_edge(v1,v3,dy_dx1); edge_count++; }
                if(x2.reachable){ add_edge(v2,v3,dy_dx2); edge_count++; }
            }
        }
    }
    check_memory();
}
```

Three things in that fragment are load-bearing.

The profiling pass allocates nothing. It only counts. That is what makes the budget decision possible before a single vertex exists, and it is what Chapter 8 exploits: the exact vertex and edge counts are known in advance.

Aliasing is handled explicitly. `&x1==&x2` detects `x*x`, where a naive implementation would add two edges between the same pair of vertices and then have to merge them. The partials are summed instead. `examples/invariants` pins this path as its `alias` case.

The NULL initialisers are a fix, not style. SVEGP-20: those pointers were left uninitialised, and `add_edge` only guards against NULL. An indeterminate non-null value walked straight into `tgt->add_in_edge(src,...)`. The library’s own callers always set the reachability flags consistently, so it never fired internally — but `binary_op` and its siblings are *exported* for hand-written operators, and there a mismatched flag turns a dropped edge into a wild pointer dereference.

2.3 Reachability

`reachable` is the flag that keeps passive arithmetic out of the graph. A value is reachable if it descends from a registered independent. A computation on two passive values records nothing: `operator=` from a `double` calls `passive_op`, which kills the vertex and sets `idx=0`. This is why a program can mix `active` and `double` freely and only pay for the part that actually depends on the inputs.

2.4 The graph

A `Vertex` is 72 bytes and holds nothing but its identity and its two adjacency lists:

```
class Vertex
{
public:
    bool    alive;           // true if this vertex is a declared dependent
    largeint idx;
    largeint owner_idx;

    Adjacency in_edges;
    Adjacency out_edges;
    ...
};
```

An `Edge` is 24 bytes — a partial derivative and its two endpoints:

```
class Edge
{
public:
    double    eval;         // the local partial derivative
    Vertex * src;
    Vertex * tgt;
    ...
};
```

`alive` deserves a note, because the name is misleading: it does not mean “not yet destroyed”. It marks a vertex as a *declared dependent*, and its only effect is that elimination refuses to eliminate it. Dependents must survive because they are the rows of the Jacobian.

`Adjacency` is a sorted array of (`neighbour index`, `edge pointer`) pairs, binary searched. Its members are called `first` and `second` because the elimination code was written against `std::map` and is full of `it->second`; keeping the names meant the representation could be replaced without touching a line of the algorithm. Chapter 8 covers how the storage beneath it evolved.

Chapter 3

Elimination

3.1 The operation

Eliminating an intermediate vertex v removes it from the graph while preserving every path through it. For each in-edge (u, v) with weight a and each out-edge (v, w) with weight b , the product ab is added to the edge (u, w) , creating it if it does not exist. The cost is the number of such products:

$$\text{cost}(v) = \deg^-(v) \cdot \deg^+(v).$$

The implementation is the whole of `Vertex::eliminate`:

```
largeint Vertex::eliminate( EdgeArena & arena , AdjArena & adj )
{
    largeint m = in_edges.size();
    largeint n = out_edges.size();
    largeint cost = m*n;

    for( outedge_it=out_edges.begin() ; outedge_it!=out_edges.end() ; outedge_it++ )
        outedge_it->second->tgt->in_edges.erase(this->idx);

    for( inedge_it=in_edges.begin() ; inedge_it!=in_edges.end() ; inedge_it++ )
    {
        inedge_it->second->src->out_edges.erase(this->idx);

        for( outedge_it=out_edges.begin() ; outedge_it!=out_edges.end() ; outedge_it++ )
        {
            double cij = (inedge_it->second->eval)*(outedge_it->second->eval);

            Edge * direct_link = NULL;
            Adjacency::iterator direct_link_it =
                inedge_it->second->src->out_edges.find(outedge_it->second->tgt->idx);

            if(direct_link_it!=inedge_it->second->src->out_edges.end())
                direct_link = direct_link_it->second;

            if(direct_link){
                direct_link->eval += cij;                // fold into an existing edge
            }else{
                outedge_it->second->tgt->add_in_edge(        // fill-in
                    inedge_it->second->src , cij , arena , adj );
            }
        }
    }
    arena.release( inedge_it->second );
}
```

```

for( outedge_it=out_edges.begin() ; outedge_it!=out_edges.end() ; outedge_it++ )
    arena.release( outedge_it->second );

in_edges.clear(adj);
out_edges.clear(adj);
return cost;
}

```

The `find` on the predecessor’s out-list is the *direct-link probe*, and it is the reason adjacency must stay sorted rather than becoming a linked list. Measured degrees on that list are 40 entries at $n_x = 20$ and 109 at $n_x = 40$, growing with problem size; seven comparisons over contiguous memory beat a tree walk, but a linear scan over 109 scattered nodes would have made elimination asymptotically worse on exactly the deep tapes the library exists for.

3.2 Order matters, and adjacency order does not

This distinction was assumed backwards during design and was settled by experiment.

Within one `eliminate()` call, `in_edges` is keyed by source index and `out_edges` by target index, so each (u, w) pair occurs exactly once and `direct.link->eval += cij` fires at most once per pair per elimination. All accumulation *across* eliminations is ordered by elimination order, which comes from `intmed_vec` and `intmed_map`, never from adjacency.

Reversing the adjacency iteration order therefore changes nothing — verified by doing it, against twelve pinned Jacobian hashes, all of which stayed byte-identical. That freed the storage layout to be chosen for speed alone.

Elimination order, by contrast, changes the floating-point result in the last bits, exactly as it should: `examples/invariants` records `nonlinear/rev` at hash `0xae4c2dad9593bc3` and `nonlinear/fwd` at `0x814891dd92fad4e7`, with costs 3291 and 3333. Same Jacobian to within rounding; different accumulation order; different last bits. Any change that reorders elimination is therefore a change to the numbers, and the pinned table will say so.

3.3 Direction

```

largeint Process::eliminate()
{
    if(elim_mode==FORWARD_ELIM){ return forward_eliminate(); }
    return reverse_eliminate();
}

```

Forward elimination walks `intmed_vec` in order; reverse walks it backwards. Both refuse to eliminate a vertex whose `alive` flag is set, and both skip vertices with no in-edges (independents) or no out-edges that are dependents.

SVEGP-23 is worth recording: the direction was not selectable at all. `checkpoint()` called `reverse_eliminate()` unconditionally, so forward elimination could never be the elimination that produces the Jacobian — calling `forward_elimination()` afterwards merely walked an already-eliminated graph and reported the same cost. The `elim_t` enum had sat unused in `Typedefs.hpp` since the beginning, and the old test Makefile recorded that a `set_elim_mode()` had once been *removed* from the API, so restoring it was recovering an intended feature rather than inventing one.

3.4 Harvest

After elimination, the surviving edges from independents to dependents are the Jacobian entries. `Process::harvest` walks the dependents, and for each looks up the edge from each independent:

```
Edge * e = dep_vPtr->from(indep_vPtr);

if(e){
    A[dep_pos][i] = e->get_partial();
}
else if(indep_vPtr==dep_vPtr){
    A[dep_pos][i] = 1.0;      // a dependent that IS an independent
}
```

The `indep_vPtr==dep_vPtr` branch handles $y = x$ — a dependent that is also an independent has no edge to itself and a derivative of exactly one. `examples/invariants` pins this as its `passthrough` case.

Two defects lived here. SVEGP-03: the column of an entry is the *position* of the independent, not its global vertex index; using `get_idx()-1` coincides with the column only while the independents happen to occupy indices $1..n$ in registration order, and registering one after any active arithmetic both lost a column and wrote past the end of the row. SVEGP-05: the requested shape used to be trusted, so a caller passing m and n transposed overran the heap. It is checked now, and refuses:

```
if( m != this->dep_vec.size() || n != this->indep_count ){
    std::cerr << "maxwell::harvest: requested " << m << "x" << n
               << " but the graph holds " << this->dep_vec.size() << "x" <<
               this->indep_count
               << " -- refusing to write out of bounds.\n";
    return;
}
```

Chapter 4

Checkpointing

4.1 The idea

A tape for a deep computation does not fit in memory. The classical remedy is to store the program's intermediate *state* at checkpoints and recompute between them. Maxwell does the dual: it stores nothing and *replays the program*, recording a different slice of the graph each time.

The user's computation is placed in a *section* driven by a loop:

```
while( checkpoint(x,y) ){
    try{ f(); }
    catch( BreakException const & ){}
```

`checkpoint` returns `true` while another pass is needed. The first pass is the *profiling* pass: nothing is allocated, everything is counted, and the partition boundaries are discovered. Then one *productive* pass per partition follows.

4.2 Partitions and the owner index

Partition membership is carried on two counters in `Process`: `next_owner_idx`, which advances as the section runs, and `tgt_owner_idx`, which names the partition this pass is to record. Recording is gated by a single scalar equality:

```
bool Process::is_proc()
{
    if(next_owner_idx==tgt_owner_idx){ return true; }
    return false;
}
```

Every vertex-creating helper begins with `if(is_proc())`. So during a productive pass the section's arithmetic runs in full, but only the operations belonging to one partition build graph. **Exactly one partition is active at a time, by construction** — a fact with consequences for parallelism that Chapter 9 returns to.

Boundaries are found by `check_memory`, called after every recorded operation:

```
void Process::check_memory()
{
    largeint cur_mem_usage = (intmed_count*VERTEX_BYTES + edge_count*EDGE_BYTES);
    largeint part_size = cur_mem_usage - prev_mem_usage;

    if(part_size>=mem_size){
        if(!profiling){
```

```

    if(is_proc()){
        tgt_owner_idx--;
        ... move intmed_vec into intmed_map, merging duplicates ...
        if(throwable){ throw BreakException(); }
    }
}
prev_mem_usage = cur_mem_usage;
next_owner_idx++;
}
}

```

When the active partition is complete, the pass has no further use for the rest of the section, so it is abandoned by throwing `BreakException` from inside the operator. This is the mechanism that makes replay cheap: a pass that has already recorded its partition does not run the remaining arithmetic at all.

The invariant `BreakExceptions = chunks - 1` holds throughout, because the first productive pass never throws: `reinitialize()` sets `throwable` only once its own run counter is non-zero.

4.3 The replay loop

```

bool maxwell::checkpoint( active * x , active * y )
{
    TapeState * tp = current_tape();
    if(!tp){ /* diagnose and end the loop */ return false; }

    Process * myProc = &tp->proc;

    if(!tp->run_counter){ tp->run_counter++; return true; } // profiling pass

    if(tp->run_counter==1){
        for( largeint k=0 ; k<tp->dependent_size ; k++ ){
            y[k].old_idx = y[k].idx;
            tp->dep_shadow_copy[k] = y[k].val;
        }
        myProc->finalize(); // close profiling
        tp->run_target = myProc->get_tgt_owner_idx();
        myProc->disable_profiling();
    }else{
        myProc->eliminate(); // fold the partition just recorded
    }

    internals::restore_values(x,y); // put the independents back
    myProc->reinitialize();

    if(tp->run_counter<=tp->run_target){ tp->run_counter++; return true; }

    for( largeint k=0 ; k<tp->dependent_size ; k++ ){
        y[k].idx = y[k].old_idx;
        y[k].val = tp->dep_shadow_copy[k];
    }
    return false;
}

```

`restore_values` resets the independents from `indep_shadow_copy`, which is why the section produces identical values on every pass. Note that `tgt_owner_idx` counts *down*: the first

productive pass records the *last* partition, and elimination accumulates backwards through the tape.

4.4 The contract, which is the sharpest edge in the library

The section is run $1 + N$ times, where N is not known until profiling finishes, and it may be aborted at an arbitrary operator by a `throw` from inside `operator*`. Three requirements follow, and for a long time none of them was written down anywhere:

1. it must be **re-runnable** — same inputs, same result, every time;
2. it must be **side-effect free** — the profiling pass runs it an extra time, so any I/O or RNG inside it happens once more than the author expects;
3. it must be **exception-safe** — a `BreakException` unwinds from the middle of the computation.

Requirement (3) was broken during a port: `heat_assim` originally held its two state buffers in raw `new[]` and swapped them by pointer, leaking $n_x + 1$ actives on *every* break — thousands across a conjugate-gradient run. It took a rewrite to `std::vector<active>` to fix, and nothing would have caught it.

The RAI surface (Chapter 7) takes requirement (3) off the caller entirely by owning the `try/catch`. It cannot take (1) and (2), and does not pretend to.

4.5 The property that makes it trustworthy

Chunking must not change the answer. `examples/regress` asserts this directly, sweeping budgets against a reference computed at 2^{30} bytes:

budget (bytes)	max relative deviation
200 000	0.0
50 000	0.0
20 000	0.0
8 000	0.0
4 000	0.0
2 000	0.0
1 000	0.0

Exactly zero, not a tolerance. `examples/invariants` adds twelve pinned witnesses across five tape shapes, each run whole and chunked: the `lattice` case gives an identical Jacobian hash whole, at 5 chunks, at 16 chunks, and under both elimination directions.

4.6 What replay costs

Measured on a synthetic tape of 16 variables and 120 steps ($\approx 1\,920$ recorded operations), GCC 13.3, -O2:

budget	partitions	time	vs. whole	replay share
the program itself, no AD	—	0.0085 ms	—	—
whole tape	1	5.78 ms	1.00×	—
200 kB	8	15.40 ms	2.66×	62%
50 kB	29	18.35 ms	3.18×	68%
12 kB	121	36.40 ms	6.30×	84%

Two conclusions. First, the differentiation machinery costs roughly $680\times$ the underlying arithmetic, so the target for any optimisation is the tape, never the user’s function. Second, the elimination cost is *identical* at every budget (33 647 multiply–adds), so all of the growth in that table is replay. Chunking is not free, and the cheapest speedup available to a user is a larger budget.

Chapter 5

The memory budget

5.1 What a graph costs

The budget is spent against a model of the graph’s payload:

```
static const largeint VERTEX_BYTES = sizeof(Vertex);           // 72
static const largeint EDGE_BYTES   = sizeof(Edge) + 2*sizeof(AdjEntry); // 24 + 32 =
56
```

used at exactly three sites: `check_memory()`, `get_memory()`, and the profiling branch of `Process::finalize()`.

5.2 svegp-24: the accounting never counted adjacency

The original model was `intmed_count*sizeof(Vertex) + edge_count*sizeof(Edge)`. But an edge is not only its `Edge` object: it is also the *two adjacency entries that reference it*, one in the target’s in-list and one in the source’s out-list. Before the arenas those were a pair of red-black tree nodes and went uncounted; after the representation change they became a pair of `AdjEntry` and were *still* uncounted. Same omission, different container. An edge costs $24 + 2 \times 16 = 56$ bytes of payload, not 24, so the budget had been under-charging every edge by more than half since the beginning.

The constants deliberately exclude allocator overhead, container slack and arena block granularity. Those are properties of the allocator, not of the graph, and — decisively — they are not knowable during the profiling pass, which is when the budget decision is made and when nothing has been allocated yet. So the model is the graph’s payload exactly, and the process always holds somewhat more.

5.3 The two errors were cancelling

Correcting the edge charge moved the chunk counts *back to where they had started*:

	<code>sizeof(Vertex)</code>	per edge	lattice chunk counts
original	120	24	5, 16, 4
after the representation work	72	24	4, 11, 3
after SVEGP-24	72	56	5, 16, 4

The original formula over-charged for vertices (120 against a real payload of 72) and charged nothing for adjacency, and the two errors roughly cancelled. That is why the examples’ hand-calibrated budgets worked at all, and why recalibrating them turned out to be almost a no-op: `bratu` 14 → 13, `hybrdj1` 20 → 18, `heat` 100 → 99.

This is the most instructive defect in the library’s history. A wrong model that happens to produce right-looking answers is harder to find than one that produces wrong answers, and it is found only by measuring the thing the model claims to predict.

5.4 The drift alarm

Because the model deliberately under-reports, the gap between it and reality must be watched rather than assumed. `examples/invariants` measures process heap against the accounted figure on two fixed shapes and fails outside a band:

```
const double LOW = 1.2, HIGH = 1.9;
...
const double ratio = double(peak)/double(acct);
const bool ok = (ratio>=LOW && ratio<=HIGH);
```

The band is set by experiment, not by guesswork, and it has been re-pinned twice as the representation changed:

	correct accounting	SVEGP-24 reverted
before the adjacency arena	1.67 – 1.70	2.38 – 2.42
after the adjacency arena	1.51 – 1.56	2.16 – 2.22

The first attempt at a band was [1.0, 3.0], and the control — reverting SVEGP-24 — walked straight through it; only the pinned chunk column caught the reverted fix. The band was tightened to [1.2, 2.2]. Then the adjacency arena lowered the real heap, which lowered *both* columns, and 2.16 landed back inside [1.2, 2.2]: the same failure, a second time. It is [1.2, 1.9] now, verified in both directions.

The alarm is skipped under AddressSanitizer, which replaces the allocator wholesale and makes `mallinfo2` report ASan’s redzoned bookkeeping rather than the program’s footprint. The first ASan run of the alarm reported “0 accounted, 0 heap, `-nanx`”.

5.5 svegp-25: a budget that was not a budget

`examples/heat` was the only example not taking a plain byte budget. Its third argument was a *multiplier*: the budget was `it*431208`, and 431 208 was one time step’s tape footprint measured at $n_x = 400$. So `it` meant “how many $n_x=400$ time steps fit in a chunk”, and at any other grid it silently meant something else — at $n_x = 40$ a step costs about a tenth as much, so `it=1` bought ten steps per chunk rather than one.

It takes bytes now, and the knob finally behaves like one:

invocation	chunks
40 100 400000	11
40 200 400000 22 <i>double the steps, double the chunks</i>	
100 400 400000	107
100 400 800000 54 <i>double the budget, half the chunks</i>	

The change had a hazard: `main.cpp` and `RUNARGS` in the example’s Makefile had to agree, and bytes in one with a stale 1 in the other is a one-byte budget, which chunks on every operation and reads as a hang rather than a mistake. So `main.cpp` rejects an implausibly small budget and names the exact fix. A stale Makefile now fails in a second with instructions instead of appearing to hang.

Chapter 6

The tape object

6.1 svegp-26: two global states that had to agree

The library used to have not one global but two, and they had to be kept consistent by hand: `Process::instance`, a singleton holding the graph; and about a dozen file-statics in `API.cpp` holding everything else about the recording — `run_target`, `run_counter`, `independent_size`, the 2-D dimensions, and the two shadow-copy buffers. Neither knew about the other.

SVEGP-17 — a stride bug that wrote past the end of the heap on any non-square grid — lived in the second one, in code that had no business existing apart from the tape it described:

```
// the row stride of a dep_y_dim x dep_x_dim grid is dep_x_dim (the COLUMNS),
// not dep_y_dim. Flattening with dep_y_dim aliased entries onto each other
// and ran off the end of dep_shadow_copy as soon as the grid was not square.
for( largeint i=0 ; i<tp->dep_y_dim ; i++ ){
    for( largeint j=0 ; j<tp->dep_x_dim ; j++ ){
        tp->dep_shadow_copy[tp->dep_x_dim*i+j] = y[i][j].val;
    }
}
```

Every example used a square grid — bratu calls `set_indep_dimension(n,n)` — which is why nothing caught it for years.

Both states are now members of one object:

```
class TapeState
{
public:
    Process proc;                // the graph, by value

    largeint independent_size, dependent_size, memory_available;
    largeint indep_x_dim, indep_y_dim, dep_x_dim, dep_y_dim;
    largeint run_target, run_counter;

    double * indep_shadow_copy;
    double * dep_shadow_copy;
    ...
private:
    TapeState( const TapeState & );           // owns buffers and a graph
    TapeState & operator=( const TapeState & );
};
```

and the only mutable object left at namespace scope in the entire library is one pointer:

```
static thread_local TapeState * g_current_tape = NULL;
```

nm on the archive confirms it: one entry in the data segment. Everything else static is `const`.

No public API changed. The operators still *find* the tape rather than being handed one, so no signature moved and no user program was touched — which is precisely what let the existing verification net check the whole change. All twelve pinned invariants came out bit-identical.

6.2 What that bought, precisely

Being exact about which limitation came from which decision was the point of the exercise:

- **Two tapes at once** — structurally impossible under the singleton, since `get_proc_instance()` handed the same `Process` to everybody. Now routine.
- **Threads** — the `thread_local` pointer removes the structural reason the library could never be thread-safe. Chapter 9 measures what that actually delivers.
- **Not nesting.** A tape whose edge weights are themselves `active` — what an exact Hessian needs — requires the scalar type to be templated. That is a separate and larger change, and removing the singleton does not begin it.

6.3 svegp-27: the compiler found what had to move, not what had to be guarded

Deleting the file-statics and letting the build fail was the right way to enumerate the work: 29 uses of `myProc` and about 40 of the rest, all named precisely. But the technique has a blind spot, and it bit.

A function reading a *file-static* was reading something that always existed and was simply zero. The same function reading `tp->field` dereferences a pointer that is null when no tape is open. The code compiles either way.

`checkpoint()` was the sharp one. With no tape it used to return `true` on the first call — `run_counter` was 0 — run the caller's section with nothing recording, then null-dereference on the *second* call. Seven entry points needed guards no compiler was going to ask for. They are guarded now, preserving the old tolerant answers where those were sensible (`get_partitions()` still returns 0 with no tape), and `checkpoint()` ends the loop before it starts rather than running a section that records nothing.

`examples/regress` now calls *every* public entry point with no tape open and asserts none of them crash. That is the test that would have caught it, and it did not exist until the bug made the case for it.

6.4 svegp-27: ownership belongs in a destructor

A second, quieter defect followed from the same change. `initialize()` on an already-open tape diagnoses the misuse and deletes the old tape — but `~TapeState` freed only the two shadow buffers, and `Process` had *no destructor at all*. The graph teardown lived only in the productive branch of `Process::finalize()`, which that path never calls. Every `Vertex` leaked, in proportion to the tape.

Reproduced under LeakSanitizer on a two-line program: 280 bytes in 6 allocations, with stacks in `vertex_on_lhs` and `vertex_on_rhs`. It scales with the graph, so a program that re-initialises in a loop leaks the whole thing each time.

The fix moved the teardown into a private `destroy_graph()` that both `~Process()` and `finalize()` call:

```

void Process::destroy_graph()
{
    std::set<Vertex*> owned;

    for( ... intmed_map ... ) if(it->second) owned.insert(it->second);
    for( ... intmed_vec ... ) if(*it)        owned.insert(*it);

    for( std::set<Vertex*>::iterator it=owned.begin() ; it!=owned.end() ; it++ )
        retire(*it);

    intmed_map.clear();
    intmed_vec.clear();
    dep_vec.clear();

    edge_arena.clear();
    adj_arena.clear();
}

```

The `std::set` is deliberate. `intmed_vec` and `intmed_map` never hold the same vertex at a moment a caller can observe — each transfer clears the source before returning — but a teardown is the wrong place to lean on that, and a graph abandoned mid-section is exactly where the two containers are least likely to be in the state the happy path leaves them in. `dep_vec` is cleared rather than deleted from: every vertex in it is also in `intmed_map`, so deleting from both would be a double free.

Chapter 7

The RAII surface

7.1 Why

The free-function protocol is eleven ordered steps over hidden global state, and — this is the part that matters — four of them go wrong by being *forgotten* rather than by being written incorrectly: the `try/catch`, `free_jacobian`, `finalize`, and the `delete[]`s. Every misuse the audit found was a silent memory error before it was made a diagnostic. Diagnostics are second best. The best is making the misuse unrepresentable:

```
Tape t(n,m,budget);           // ctor/dtor replace initialize/finalize
active * x = t.independents(x0); // hands back the only thing you can use
t.run( y , [&]{ y = f(x); } ); // owns the loop AND the try/catch
t.dependents(y);
Jacobian J = t.harvest();      // a value; no free_jacobian
```

Each step returns or requires the thing the next one needs, so a step cannot be skipped or reordered without the compiler saying so.

7.2 A wrapper, not a second implementation

This is the central implementation decision of the surface. Every member makes its tape current for the duration and then calls the same free function the examples have always called:

```
maxwell::Jacobian maxwell::Tape::harvest( bool print_out )
{
    Scope s(*this);

    const largeint m = st_>dependent_size;
    const largeint n = st_>independent_size;

    double ** A = NULL;
    maxwell::harvest(m,n,A,print_out);

    if(!A) return Jacobian();

    Jacobian J(m,n);
    for( largeint i=0 ; i<m ; i++ )
        for( largeint j=0 ; j<n ; j++ ) J(i,j) = A[i][j];

    maxwell::free_jacobian(m,A);
    return J;
}
```

Consequently the twelve pinned invariants, the 68 finite-difference checks and the budget-invariance sweep all cover the new surface, because underneath it *is* the old one. A reimplementa-tion would have needed its own net. The claim was then tested directly by migrating the verification harness itself to the object surface: all twelve rows came out bit-for-bit identical.

7.3 Scopes

A `Tape` is not current outside its own calls. That is what lets two `Tape` objects coexist without either being closed, and what makes a throw out of a caller’s section leave the current-tape pointer where it found it:

```
class Scope
{
public:
    explicit Scope( Tape & t ) : t_(t) { t_.enter_(); }
    ~Scope(){ t_.leave_(); }
private:
    Tape & t_;
    Scope( const Scope & );
    Scope & operator=( const Scope & );
};
```

7.4 svegp-29: a bug only a constructed test could find

`enter_/leave_` originally kept a single saved-tape slot. That is correct for nesting two *different* tapes, and wrong for a tape re-entered from inside its own section — which happens the moment a caller reads `t.memory()` while recording, as the drift alarm does:

```
void maxwell::Tape::enter_()
{
    if( st_->scope_depth==0 ){ st_->saved = internals::current_tape(); }
    st_->scope_depth++;
    internals::set_current_tape(st_);
}

void maxwell::Tape::leave_()
{
    if( st_->scope_depth ) st_->scope_depth--;
    if( st_->scope_depth ) return;           // still inside an outer scope
    internals::set_current_tape( st_->saved );
    st_->saved = NULL;
}
```

Without the depth counter the inner scope overwrites the real predecessor with the tape itself, and leaving the *outer* scope restores `NULL` rather than the enclosing tape. Nothing reports it: an enclosing section simply carries on recording onto no tape at all and harvests a silently wrong Jacobian.

What is instructive is that the example which *contains* the re-entrant call does not detect the bug — it has no enclosing tape for the clobber to destroy. Only a deliberately constructed case in the regression suite, with one tape running inside another’s section, fails. The migration created the hazardous call; a test written specifically for it revealed the consequence.

7.5 Mixing the two surfaces

Since the free functions remain public, a program could call `finalize()` from inside a `Tape` section — which would delete the state the `Tape` object still points at, a use-after-free the moment the block ends. Both `initialize()` and `finalize()` refuse when the current tape is object-owned:

```
if( open && open->owned ){
    std::cerr << "maxwell::initialize: a maxwell::Tape object is recording -- "
                "not opening a second tape over it.  Use one surface or the "
                "other, not both.\n";
    return;
}
```

Removing that guard and running the suite under AddressSanitizer produces `heap-use-after-free`, which is how the guard is known to be load-bearing.

7.6 Why the free functions stayed

The obvious follow-up to an object surface is to retire what it wraps. That was considered and rejected on evidence: three things the object surface cannot express, all of them exercised by the regression suite.

- **Stepping the loop.** `run()` owns the whole checkpoint loop, which is exactly what makes it safe — the `try/catch` cannot be forgotten. The price is that a caller cannot interleave two tapes mid-replay, which the strongest available test of the two-tape capability does.
- **Testing the lower layer.** The free functions are still public and are what every `Tape` call turns into. Tests of their misuse behaviour are not obsolete; they are the only thing holding it.
- **Cross-checking.** The suite compares the object path against the free-function path bit-for-bit. Migrating the reference would make that comparison compare the wrapper with itself.

So the answer is that they stay, as a documented lower layer.

Chapter 8

Storage: the two arenas

8.1 Where the memory went

The original graph representation was two `std::map<largeint, Edge*>` per vertex. That cost 96 of the vertex's 120 bytes before a single edge existed, and every edge bought *three* allocations: one for the `Edge` itself and two red-black tree nodes, one in each direction, each with the allocator's own header, scattered across the heap.

Measured on one `heat_assim` tape ($n_x=20$, $n_t=40$):

what <code>get_memory()</code> accounted	728 856 bytes
what was actually on the heap	1 780 656 bytes
ratio	2.44×

So the memory budget — the feature the whole checkpointing design exists to serve — was wrong by a factor of two and a half. That is also why the budgets in the examples were magic numbers found by experiment: the model did not predict reality, so someone measured instead.

The representation was replaced in three steps, each verified against the pinned invariants before the next began.

8.2 Step 1: the edge arena

Edges come from blocks and freed slots go on a free list, with the adjacency maps left exactly as they were. The smallest change that removes one of the three allocations per edge.

```
Edge * EdgeArena::acquire( Vertex * src , Vertex * tgt , double eval )
{
    Edge * e = 0;

    if( !free_list.empty() ){
        e = free_list.back();
        free_list.pop_back();
    }else{
        if( blocks.empty() || block_used==BLOCK ){
            blocks.push_back( new Edge[BLOCK] );
            slots += BLOCK;
            block_used = 0;
        }
        e = blocks.back() + block_used;
        block_used++;
    }
}
```

```

e->eval = eval; e->src = src; e->tgt = tgt;
live++;
return e;
}

```

Recycling was already the behaviour: `eliminate()` releases an in-edge and then allocates fill-in inside the same loop, so the allocator could always have handed back the block it had just freed. The free list makes that explicit rather than introducing it — which is why the pinned Jacobians did not move.

	before	after
allocations per tape	39 462	27 908
peak heap	1 780 720 B	1 559 216 B
under-count ratio	2.44×	2.14×

The 11 554 allocations that disappeared are exactly the per-edge `new Edge`. What remained — about 23 000 tree nodes, two per edge — confirmed the diagnosis: *the map nodes were the bulk, not the `Edge` objects*.

8.3 Step 2: sorted adjacency

The two `std::maps` became one sorted array per vertex per direction. The choice of a sorted array over a linked list was made on the degree measurements: the direct-link probe scans a predecessor’s out-list of 40 to 109 entries, growing with problem size, so the search had to stay logarithmic.

	original	after step 1	after step 2
allocations per tape	39 462	27 908	17 941
peak heap	1 780 720	1 559 216	820 896
<code>sizeof(Vertex)</code>	120	120	72
under-count ratio	2.44×	2.14×	1.63×

Peak memory down 54% and allocations down 55% from the start. All twelve *structural* pinned values — every Jacobian hash and every elimination cost — were byte-for-byte identical. The *accounting* column moved exactly as predicted, 5 → 4, 16 → 11, 4 → 3 chunks, because `sizeof(Vertex)` fell and the same byte budget therefore accounted for more graph. That split between structural and accounting values is what let a single test report “the derivative is unchanged and the partitioning moved” rather than simply “failed”.

8.4 Step 3: the adjacency arena

The last per-vertex allocation. A `std::vector` adjacency was one allocation per vertex per direction the moment it became non-empty, plus a reallocation on every doubling, plus the doubling’s slack — and elimination creates and destroys vertices in the millions.

This step had to wait for the tape object, and the reason is instructive. An arena needs somewhere to put the *release*. A `Vertex` has no arena and giving it a back-pointer would have cost eight of the seventy-two bytes the arena exists to save. So `Process` owns the arena and is the only thing that destroys a vertex:

```

void Process::retire( Vertex * v )
{

```

```

    if(!v) return;
    v->in_edges.clear(adj_arena);
    v->out_edges.clear(adj_arena);
    delete v;
}

```

and `~Vertex()` is deliberately empty, with a comment saying so, because an empty destructor that looks like an oversight is worse than no destructor at all.

The arena carves fixed size classes from slabs and recycles them per class:

```

AdjEntry * AdjArena::acquire( largeint want , largeint & cap )
{
    const largeint c = class_of(want);
    cap = cap_of(c);
    ...
    if( !recycled[c].empty() ){
        AdjEntry * p = recycled[c].back();
        recycled[c].pop_back();
        return p;
    }

    if( left[c] < cap ){
        largeint slab = next_slab[c] ? next_slab[c] : SLAB_MIN;
        if( slab < cap ) slab = cap;
        AdjEntry * p = new AdjEntry[size_t(slab)];
        slabs.push_back(p);
        entries += slab;  cursor[c] = p;  left[c] = slab;
        next_slab[c] = (slab<SLAB_MAX) ? slab*2 : SLAB_MAX;
    }

    AdjEntry * p = cursor[c];
    cursor[c] += cap;  left[c] -= cap;
    return p;
}

```

Two parameter choices, both measured wrong first

This is worth recording in full, because both wrong choices looked right.

Flat 4 096-entry slabs. The first attempt. It hit the target — allocations fell 73% — and pushed *peak heap up* by 20 to 32%. A size class holding three blocks paid for all 64 kB of its slab. Replaced with geometric growth per class.

Geometric growth barely helped , which was the useful surprise: the real cost was the *minimum block size*. Classes started at four entries. But most vertices on a tape have in-degree one or two and out-degree one — an elementary operation has one or two operands — so the commonest allocation in the library was overshooting by two to four times, on the two blocks that *every* vertex carries. Capacities start at one entry now, and that is where the win came from.

Neither would have been found by reasoning about it. The first looked correct and was a regression; the second was invisible until the first was fixed.

Result

shape	allocations		peak heap
12 vars $\times$ 40 steps	9 297 $\rightarrow$ 2 564 (-72%)	425 680 $\rightarrow$ 362 640 (-15%)	
20 vars $\times$ 25 steps	9 766 $\rightarrow$ 2 707 (-72%)	639 040 $\rightarrow$ 595 360 (-7%)	
20 vars $\times$ 40 steps	15 467 $\rightarrow$ 4 213 (-73%)	876 896 $\rightarrow$ 811 568 (-7%)	

`sizeof(Vertex)` is still 72 — Adjacency is a pointer and two counters, the same 24 bytes a `std::vector` occupied — so `VERTEX_BYTES` is unchanged and *no chunk count moved*.

The effect on wall time was larger than the memory figures suggest and was not anticipated. Reproducibly, across three runs:

	before	after
whole tape	13.8 – 15.4 ms	5.4 – 5.7 ms
chunked, 200 kB	22.3 – 23.5 ms	14.7 – 15.2 ms

That is $2.5\times$ on an allocation-heavy tape. It is a reminder that allocation count, not resident bytes, was the dominant cost all along.

Chapter 9

Concurrency

9.1 What is shared

One object at namespace scope in the whole library is mutable, and it is `thread_local`. Everything else static is `const`: two size constants, one mathematical constant, and the arena class parameters. Every piece of state a recording touches — the graph, both arenas, the counters, the shadow buffers, the 2-D dimensions — lives in the `TapeState` that SVEGP-26 collected them into.

9.2 What works

Independent tapes on separate threads. Verified under ThreadSanitizer:

- 4 threads $\times$ 25 tapes each, half of them chunking hard so `BreakException` unwinds on several threads simultaneously: no races, every derivative correct.
- 8 threads $\times$ 6 rounds, each round building a deep tape three ways — object surface whole, object surface chunked at 3 000 bytes, and the free functions at 4 000 — and cross-checking all three: zero disagreements, no races. The free functions are as thread-safe as the object surface, because they go through the same thread-local pointer.
- **Negative control:** one `Tape` object driven from two threads races immediately, reported in `Tape::enter_()` and `independents()`. The boundary is where it should be, and the sanitizer says so loudly rather than corrupting quietly.

Measured speedup over 16 independent tapes on 2 cores:

budget	serial	parallel	speedup
whole tape	90.6 ms	50.7 ms	1.79 $\times$
200 kB	247.3 ms	129.0 ms	1.92 $\times$
50 kB	308.4 ms	157.4 ms	1.96 $\times$
12 kB	561.7 ms	281.1 ms	2.00 $\times$

Efficiency improves slightly at tighter budgets, because replay is pure per-thread arithmetic while recording contends on the allocator.

The memory trade is milder than it first appears. With a fixed 200 kB pool: one thread at 200 kB is 15.4 ms per tape; two threads at 100 kB each is about 16.4 ms per tape, two at a time, for an effective 8.2 ms — 1.88 $\times$. Roughly 94% of two cores survives halving the budget.

9.3 What does not work, and why

Parallelising *across partitions* is not available, and the obstacle is architectural rather than mechanical. Three reasons, worst first.

Value dependency. Partition k 's inputs are the program state after partitions $1..k-1$ have executed. The replay design exists precisely because that state is never stored. Building partitions concurrently means checkpointing the *values* at boundaries — which is the “spill the tape rather than replay the program” alternative, a different design with different costs, not a parallelisation of this one.

Elimination order. Each pass folds its partition into the accumulating result, last to first, each step consuming what came before. Chapter 3 showed that elimination order changes the result in the last bits. Reordering or interleaving partitions therefore changes the Jacobian, destroying the bit-exactness across budgets that has twelve pinned witnesses — which is the property that makes chunking trustworthy in the first place.

Shared recording state. `next_vertex_idx`, `intmed_vec`, the partition-boundary logic and both arenas are per-tape, not per-thread. Lockable — but locking makes index assignment nondeterministic, which returns to the previous paragraph.

The same three apply to placing an OpenMP loop *inside* a section. Making that work would need per-thread sub-tapes with index ranges reserved during the profiling pass — which already counts everything, so it could hand out deterministic ranges — followed by a merge. That is a substantial design, and it only pays if the section contains genuinely independent work, which a deep time-stepping tape does not.

One avenue remains open. Recording partition $k+1$ and eliminating partition k are order-independent: recording needs values, not the elimination result. A two-stage pipeline would preserve the accumulation chain. Its ceiling is the smaller of the two phases, which has not been measured, and it would require the arenas to be split per stage. It is noted here as the one form of intra-tape parallelism the architecture does not forbid.

9.4 A caveat on diagnostics

Diagnostics go to `std::cerr` unsynchronised. That is not a data race — the standard stream objects are safe — but two threads misusing the API simultaneously will interleave their messages mid-line.

Chapter 10

Verification

The way this library is tested is itself a design choice, and the report would be incomplete without it.

10.1 Structural values and accounting values

`examples/invariants` freezes what the library computes across twelve cases and five tape shapes, and pins *two kinds* of number:

- **Structural** — the elimination cost, and a bit-exact FNV-1a hash of every harvested Jacobian entry. These are properties of the graph and the arithmetic. They must never move.
- **Accounting** — the partition count. This is *expected* to move when the memory model changes, and is pinned so that the move is deliberate.

The split earns its keep every time the representation changes. A single-column test would have reported a failure at step 2 of the arena work and told you nothing about whether the derivative had changed.

The hash is a refactoring change-detector, not a portable golden value, so the example compiles with `-ffp-contract=off`: otherwise the compiler may fuse `a*b+c` into an FMA depending on the optimisation level, and the hash would track `-0` rather than the library.

10.2 Controls

A test that has never failed is not known to work. Every assertion added during this work was checked by deliberately breaking the thing it guards:

control	observed
reverse the out-edge iteration order	nothing — correctly
drop fill-in edges below 10^{-12}	1 case, on <i>cost</i>
perturb every partial by 1 ulp	all 12, on <i>hash</i>
revert SVEGP-24 (uncount adjacency)	chunk column, and the drift band
<code>~Process()</code> emptied	LeakSanitizer, 560 bytes in 12 allocations
<code>run()</code> stops catching <code>BreakException</code>	abort, exit 134
object-owned guard removed from <code>finalize()</code>	ASan <code>heap-use-after-free</code>

control	observed
<code>leave_()</code> stops restoring	the nested-tape case fails
<code>set_current_tape</code> made a singleton again	all three two-tape assertions fail
off-by-one in the arena's block capacity	ASan heap-use-after-free

Two entries in that table are the most valuable. The first — reversing adjacency order and observing *nothing* — is a finding, not a null result: it disproved a constraint the design had assumed and freed the storage layout. And one control that was run *did not* fail: removing the current-tape restore from `~Tape()` breaks nothing, because every scope pairs its enter with its leave even while an exception unwinds. That line stays as belt-and-braces, and the code comment says plainly that the suite does not cover it. Recording a control that passed is as useful as recording one that failed.

10.3 Sanitizers

`make sanitize` rebuilds into a *separate* tree under AddressSanitizer and LeakSanitizer. The separate tree is deliberate: `make` compares timestamps only, so sanitizing in place leaves every binary newer than its source and a following plain `make` rebuilds nothing — you keep running instrumented code without being told, which is how a fast example comes to look like a hang.

10.4 What the suite asserts

example	what it holds
<code>invariants</code>	12 pinned structural values; the accounting drift band
<code>fdcheck</code>	68 finite-difference checks of every elementary function
<code>regress</code>	every fixed defect, both API surfaces, budget invariance
<code>bratu</code>	the 2-D checkpoint overload; asserts chunking
<code>heat</code>	a 107-partition tape; asserts chunking
<code>heat_assim</code>	adjoint vs finite differences; elimination direction; CG
<code>hybrdj1</code>	a full 20×20 Jacobian; asserts chunking
<code>pinn</code>	2 502 tapes; chunked gradient equals unchunked reference
<code>test_xor</code>	4 001 tapes; AD gradient vs central differences

Part II

Using the Library

Chapter 11

Getting started

11.1 Building

```
make           # the library and every example, into build/
make test      # build, then run the whole suite in order
make list      # the examples and the arguments they run with
make sanitize  # rebuild into build-asan/ under ASan+LSan, run the suites
make clean     # remove build/ and build-asan/
```

Everything generated lands under `build/`: `build/lib/libmaxwell.a`, `build/obj/`, and one binary per example in `build/bin/`. Nothing is written into the source tree.

A single example builds standalone from a cold tree — `cd examples/pinn && make` recurses into `src/` first — and `cd src && make` builds the library on its own.

11.2 Including it

```
#include "maxwell.hpp"

using namespace maxwell;    // or qualify: maxwell::active, maxwell::sin
```

The header does not open the namespace for you. It brings in the `active` scalar, the free-function API, `Tape` and `Jacobian`, and `BreakException` — and deliberately nothing internal, so the graph representation can change underneath you without your code changing. It has, twice.

11.3 The shape of a program

```
#include "maxwell.hpp"
using namespace maxwell;

int main()
{
    const double x0[2] = { 1.1 , 2.3 };

    Tape t( 2 , 1 , 1u<<20 );           // 2 independents, 1 dependent, 1 MB budget

    active * x = t.independents(x0);    // the tape's own, values set and registered
    active y;

    t.run( y , [&]{ y = sin(x[0]*x[1]); } );
```

```
t.dependent(y);  
  
Jacobian J = t.harvest();  
  
std::printf("dy/dx0 = %.10f\n", J(0,0));  
return 0;  
}
```

Five calls. The constructor opens the tape and the closing brace closes it, on every path out of the block including a `return` or a `throw`.

Chapter 12

The tape surface

12.1 Construction

```
Tape t( n , m , memory_budget );
```

n independents, m dependents, and a budget *in bytes*. The budget is what the library may hold as graph payload; see Section 14 for how to choose it. A **Tape** is not copyable — it owns a graph.

12.2 Independents

```
active * independents( const double * x0 );  
active ** independents( largeint rows , largeint cols , const double * x0 );
```

The tape hands back its own independents, with the values already in them and already registered. This replaces three steps that had to agree with each other — declare, set values, register in the same order — with one, and it closes the bounds error that SVEGP-18 was about.

The storage belongs to the tape and is sized exactly once, so the reallocating `std::vector<active>` hazard of Section 16 cannot happen to it. The returned pointer is valid until the **Tape** dies; do not `delete` it.

The 2-D form takes `rows` and `cols` in the order the indices are written, `x[i][j]` with i over rows, and reads `x0` row-major. The underlying `set_indep_dimension` takes them the other way round, columns first; that transposition is SVEGP-17, it is invisible on a square grid, and it is now made inside the library where a caller cannot get it wrong.

12.3 Running the section

```
template<class Body> void run( active & y , Body body ); // one dependent  
template<class Body> void run( active * y , Body body ); // m dependents  
template<class Body> void run( active ** y , Body body ); // a rows x cols grid
```

`run` drives the checkpoint loop to completion and owns the `try/catch`. Call it after `independents()` and before `dependents()`. For the 2-D form, call `dependent_shape(rows,cols)` first — `checkpoint` reads the dependent shape on its first pass, so it cannot be inferred later.

12.4 Dependents, harvest, and the rest

```

void dependent ( const active & y );           // one
void dependents( const active * y );          // m of them
void dependents( active ** y );               // the dependent_shape() grid

Jacobian harvest( bool print_out = false );

largeint partitions();                        // how many chunks; 1 means it never chunked
largeint cost();                              // elimination cost, in multiply-adds
largeint memory();                            // graph payload in bytes, by the budget's model
void      set_elim_mode( elim_t mode );       // before run(); default REVERSE_ELIM

```

12.5 The Jacobian

```

class Jacobian
{
public:
    largeint rows() const;
    largeint cols() const;
    bool      empty() const;                // harvest() refused
    double    operator()( largeint i , largeint j ) const;
    double & operator()( largeint i , largeint j );
    ...
};

```

A value, stored as one flat row-major block. Copying one copies it; there is nothing to free. `empty()` is true when `harvest()` refused — a shape mismatch, or a program that never drove `run()` — and it is worth checking, because it is the one way the call can decline.

Chapter 13

The checkpoint contract

This chapter is short and is the most important one in Part II.

The code inside `run()` is executed $1 + N$ **times**, where N is the number of partitions and is not known until the first pass has finished. It may also be **abandoned part-way** by an exception thrown from inside an arithmetic operator. Your section must therefore be:

Re-runnable. Same inputs, same result, every time. The library restores the independents between passes; anything else your section reads must not have changed.

Side-effect free. The profiling pass runs your section an extra time. Printing from inside a section prints $N + 1$ times. Drawing a random number inside a section draws a different one on each pass and silently breaks re-runnability — put the RNG *outside*, as `examples/pinn` does.

Exception-safe. `run()` owns the `try/catch`, so you cannot forget it or catch the wrong thing — but an exception still unwinds through *your* code. Anything held in a raw pointer between the start of the section and the throw point is leaked. Use containers, or let the section own nothing:

```
// WRONG: leaks nx+1 actives on every partition boundary
active * old = new active[nx+1];
...
delete [] old;                                // never reached when BreakException fires

// RIGHT
std::vector<active> old(nx+1);                  // sized once; destructor runs while unwinding
```

That exact defect — raw `new[]` buffers swapped by pointer, leaking on every break, thousands of times across a conjugate-gradient run — is a real one from this library's history, and nothing would have caught it but a sanitizer.

Chapter 14

Choosing a budget

14.1 What the number means

The budget is bytes of *graph payload*: vertices at `sizeof(Vertex) = 72` and edges at `sizeof(Edge) + 2 × sizeof(AdjEntry) = 56`. It deliberately excludes allocator overhead and container slack, because those are not knowable during the profiling pass, when the decision is made. Expect the process to hold roughly 1.5 to 1.9 times the budget — `examples/invariants` measures that ratio on every run and fails if it drifts.

14.2 How to pick one

Start with the memory you can afford, divided by about 1.7. Then check what you got:

```
Jacobian J = t.harvest();
std::printf("tape broken into %lu chunk(s)\n", (unsigned long)t.partitions());
```

`partitions()==1` means the budget was never reached and the checkpoint machinery never fired. That is the right answer for a small problem, and a warning sign for a large one — it means you are holding the whole tape.

14.3 What chunking costs

From Section 4.6: 8 partitions cost $2.7\times$ the whole-tape time, 29 cost $3.2\times$, 121 cost $6.3\times$. The elimination work is identical in every case; the growth is entirely the replay of your section.

So the ordering of remedies for a slow differentiation is:

1. raise the budget if the memory exists — this is by far the largest lever;
2. choose the elimination direction (Section 14.4);
3. run independent tapes on separate threads (Section 15.6), worth up to the core count, and slightly less because each tape then gets a smaller share of memory.

14.4 Elimination direction

```
Tape t(n,m,budget);
t.set_elim_mode(REVERSE_ELIM);           // the default
// t.set_elim_mode(FORWARD_ELIM);
```

Must be set before `run()`. Both directions produce the same Jacobian — to within the last bits, since they accumulate in different orders — at very different cost. Reverse wins on a deep narrow tape, and the gap grows with depth: `examples/heat_assim` measures 6835 multiply-adds reverse against 112102 forward, a $16.4\times$ ratio, on the same graph. Forward can win when there are few independents and many dependents. `cost()` reports the figure, so the honest way to choose is to measure both once on a representative problem.

Chapter 15

Use cases

Six worked cases, each taken from a shipped example that runs under `make test`.

15.1 A gradient: many inputs, one output

The commonest case in optimisation and machine learning. With one dependent, the $1 \times n$ Jacobian *is* the gradient.

```
static double gradient( const double * w , double * g , largeint mem )
{
    Tape t( largeint(NP) , 1 , mem );

    active * p = t.independents(w);
    active L;

    t.run( L , [&]{ L = mse(p); } );

    t.dependent(L);

    Jacobian J = t.harvest();
    if(J.empty()){ return 0.0; }

    for( int i=0 ; i<NP ; i++ ) g[i] = J(0,i);

    return passive_value(L);          // the loss, as a plain double
}
```

From *examples/test_xor*, which trains a 2-4-1 network on XOR for 4 001 epochs, one tape per epoch, 8 partitions each. Note what the signature says: parameters go in as `double` and come back as `double`. They are active only *inside* the tape. That matters — see Section 16.

The gradient is checked against central differences on an independently written plain-`double` forward pass, which is the only way such a check proves anything: if both sides were the same code, agreement would be meaningless.

15.2 A full Jacobian: n inputs, n outputs

```
Tape t( n , n , budget );

std::vector<double> x0(n,-1.0);
active * x = t.independents( &x0[0] );
```

```

active * fvec = new active[n];

t.run( fvec , [&]{ fcn(n,x,fvec); } );

t.dependents(fvec);

Jacobian J = t.harvest(true);    // true keeps the FJAC listing

const double want = 3.0 - 4.0*(-1.0);
for( int k=0 ; k<n ; k++ ) assert( J(k,k)==want );

```

From *examples/hybrdj1*, MINPACK's *hybrj1* test function at $n = 20$, tape broken into 18 chunks. The dependents here are the caller's own array, not the tape's — only the independents come from the tape.

15.3 A two-dimensional grid

When the natural indexing is $x[i][j]$, the tape can hand out a grid:

```

std::vector<double> x0(n*n,0.0);
... fill x0 row-major, boundary conditions and all ...

Tape t( n*n , n*n , (largeint)bs );

active ** x = t.independents( n , n , &x0[0] );
t.dependent_shape( n , n );

t.run( x , [&]{ bratu(n,x,r); } );    // x is both independent and dependent
t.dependents(x);

```

From *examples/bratu*, a 2-D Bratu discretisation on a 50×50 grid at a 400kB budget, 13 chunks. The independents are also the dependents here, which is legal and common for an iterated map.

This is the shape SVEGP-17 lived in. Because *bratu* is square, it could never have exposed the bug; the regression suite drives a deliberately non-square 3×2 grid for that reason.

15.4 An adjoint: data assimilation

A deep tape — the whole point of the library.

```

static double gradient( const double * w , const double * obs , double * g ,
                       elim_t mode , largeint mem )
{
    Tape t( largeint(NP) , 1 , mem );
    t.set_elim_mode( mode );

    active * p = t.independents(w);
    active Jc;

    t.run( Jc , [&]{ Jc = cost_active(p,obs); } );

    t.dependent(Jc);

    const double value = passive_value(Jc);

    Jacobian J = t.harvest();
}

```

```

for( int i=0 ; i<NP ; i++ ) g[i] = J(0,i);

return value;
}

```

From *examples/heat Assim*: a 1-D heat equation marched 40 steps, with the initial condition as the control and 36 observations as data. The cost functional’s gradient is the adjoint. The example then drives linear conjugate gradient with exact Hessian-vector products obtained by differencing the AD gradient, and recovers the initial condition to an rms error of 1.0×10^{-13} over 19 controls, taking the cost from 7.71 to 3.6×10^{-29} .

The tape here is 40 time steps chained end to end and is eliminated whole rather than per partition — which is what makes the 16.4× reverse-versus-forward ratio visible.

15.5 Verifying a derivative

Never trust an AD result you have not checked once. The pattern:

```

Tape t(1,1,1u<<30);
active * x = t.independents(&x0);
active y;
t.run( y , [&]{ y = f(x[0]); } );
t.dependent(y);
Jacobian J = t.harvest();
const double ad = J.empty() ? NAN : J(0,0);

const double h = 1e-6;
const double fd = ( f_passive(x0+h) - f_passive(x0-h) ) / (2.0*h);

assert( std::fabs(ad-fd)/(1.0+std::fabs(fd)) < 2e-5 );

```

From *examples/fdcheck*, which does this for every elementary function the library overloads — 68 checks, zero failures. Use a generous tolerance: you are checking the AD, and the finite difference is the inaccurate side.

15.6 Independent tapes in parallel

Each thread builds its own tape. Nothing is shared, and no library call is needed to arrange it:

```

#pragma omp parallel for schedule(static)
for( int i=0 ; i<NCASE ; i++ ){
    Tape t( n , 1 , budget );
    active * x = t.independents(&starts[i*n]);
    active y;
    t.run( y , [&]{ y = f(x); } );
    t.dependent(y);
    Jacobian J = t.harvest();
    for( int j=0 ; j<n ; j++ ) out[i*n+j] = J(0,j);
}

```

Measured 1.79 to 2.00× on two cores, verified race-free under ThreadSanitizer. Natural fits: finite-difference Hessians (n independent gradient evaluations), parameter sweeps, multi-start optimisation, ensembles, and verification suites.

The one rule. A Tape object belongs to one thread. Sharing one across threads races immediately — ThreadSanitizer reports it in `Tape::enter_()` — and there is no locking mode

that makes it safe, because index assignment would become nondeterministic and the Jacobian would stop being reproducible.

And the caveat. P parallel tapes need P times the memory. Checkpointing exists to bound memory, so this trades back some of what it buys. The trade is mild — halving the budget to double the threads keeps about 94% of the second core — but it is real.

Chapter 16

Pitfalls

16.1 active is not a value type

Copying one records a vertex and an edge. Usually that is invisible. It stops being invisible in a container that reallocates:

```
// 5.5x the tape it should be: 15 reallocation copies, each recording
std::vector<active> h;
for( int i=0 ; i<8 ; i++ ) h.push_back( something(i) );

// fine: sized once, no element ever copied
std::vector<active> h(8);
for( int i=0 ; i<8 ; i++ ) h[i] = something(i);
```

The derivative is correct either way. The tape is five and a half times bigger, which changes how many partitions you get and how long it takes, and nothing warns. *Size your containers once, or reserve.*

16.2 Do not hold actives across tapes

An `active` that outlives its tape carries an index into a graph that has been given back. Keep long-lived state — optimiser parameters, iterates, anything that survives a tape — as `double`, and let `t.independents()` make it active for the duration:

```
// WRONG
active * p = new active[NP];
for( int e=0 ; e<epochs ; e++ ){ gradient(p,g,mem); for(...) p[i] = w[i]; }

// RIGHT
double * w = new double[NP];
for( int e=0 ; e<epochs ; e++ ){ gradient(w,g,mem); for(...) w[i] -= lr*g[i]; }
```

Three shipped examples did the first thing and were changed to do the second.

16.3 Do not mix the two surfaces

Calling `finalize()` from inside a `Tape` section would free the state the object still points at. The library refuses and says so, rather than letting it happen — but the diagnostic means your program is doing something it should not.

16.4 The budget is bytes

Not kilobytes, not a multiplier, not a number of time steps. One example historically took a multiplier and it silently meant a different thing at every grid size; it takes bytes now, and rejects an implausibly small value rather than chunking on every operation and appearing to hang.

16.5 Check empty()

`harvest()` returns an empty `Jacobian` rather than crashing when it refuses — a transposed shape, or a program that never drove `run()`. If you index it without checking, you get zeros rather than a diagnostic.

Appendix A

Defect log

Defects found and fixed during the 2026 audit, referred to throughout by number. Numbering starts at 16; earlier numbers belong to the pre-audit history.

id	summary
SVEGP-03	harvest wrote the Jacobian column at the independent's vertex index rather than its position
SVEGP-04	harvest did nothing while still profiling, leaving the caller's pointer uninitialised
SVEGP-05	the requested Jacobian shape was trusted; a transposed request overran the heap
SVEGP-06	harvest printed unconditionally; <code>printf</code> with a non-literal format and <code>largeint</code> in <code>%4d</code>
SVEGP-07	no way to release the harvested <code>double**</code> ; every harvest leaked
SVEGP-16	<code>operator>=</code> declared but never defined — a link failure
SVEGP-17	the 2-D checkpoint flattened rows with the wrong stride; heap overflow on any non-square grid
SVEGP-18	<code>independent()/dependent()</code> unguarded; null deref and heap overflow
SVEGP-19	<code>forward_elimination()/reverse_elimination()</code> never eliminated anything
SVEGP-20	uninitialised <code>Vertex*</code> handed to <code>add_edge()</code>
SVEGP-21	<code>Process::mem_size</code> never initialised
SVEGP-22	the partition count was not observable from the public API
SVEGP-23	the elimination direction was not selectable
SVEGP-24	the memory accounting never counted adjacency; every edge under-charged by more than half
SVEGP-25	one example's budget was a multiplier of a stale, undocumented unit
SVEGP-26	two parallel global states that had to agree; replaced by one tape object
SVEGP-27	<code>Process</code> had no destructor; a tape released other than through <code>finalize()</code> leaked its graph
SVEGP-28	the RAI surface: <code>Tape</code> and <code>Jacobian</code>
SVEGP-29	a tape re-entered from inside its own section restored the wrong predecessor
SVEGP-30	the adjacency arena

Still open by choice: SVEGP-10, an API call-order state machine that was written and never wired in. An out-of-order call is diagnosed but not made unrepresentable, except where the `Tape` surface makes it so.

Appendix B

Measurement summary

All figures GCC 13.3.0, -O2, x86-64.

B.1 Graph representation

	original	edge arena	sorted adjacency	adjacency arena
allocations / tape	39 462	27 908	17 941	−72% further
peak heap (bytes)	1 780 720	1 559 216	820 896	−7 to −15% further
<code>sizeof(Vertex)</code>	120	120	72	72
accounting ratio	2.44×	2.14×	1.63×	1.51–1.56×

B.2 Cost of chunking

16 variables, 120 steps, ≈1 920 recorded operations.

budget	partitions	time	replay share
no AD at all	—	0.0085 ms	—
whole tape	1	5.78 ms	—
200 kB	8	15.40 ms	62%
50 kB	29	18.35 ms	68%
12 kB	121	36.40 ms	84%

B.3 Parallel independent tapes, 2 cores

budget	serial	parallel	speedup
whole tape	90.6 ms	50.7 ms	1.79×
200 kB	247.3 ms	129.0 ms	1.92×
50 kB	308.4 ms	157.4 ms	1.96×
12 kB	561.7 ms	281.1 ms	2.00×

B.4 Suite status

example	result
<code>invariants</code>	PASS — 12 structural values pinned; drift $1.51\text{--}1.56\times$
<code>fdcheck</code>	0 failures of 68
<code>regress</code>	PASS, 0 failures
<code>bratu</code>	13 chunks
<code>heat</code>	107 chunks
<code>heat_assim</code>	PASS — rms 1.001×10^{-13} over 19 controls
<code>hybrdj1</code>	18 chunks
<code>pinn</code>	PASS — 2502 tapes
<code>test_xor</code>	PASS — 4001 tapes, 8 chunks each

Zero warnings from a clean build under `-Wall -Wextra`; clean under AddressSanitizer and LeakSanitizer.

Appendix C

Known limitations

No second derivatives. A Hessian requires a tape whose edge weights are themselves `active`, which requires the scalar type to be templated. This is the largest change remaining and is not begun. Where an exact Hessian is needed today, difference the AD gradient — `examples/heat_assim` does exactly that, and its symmetry check agrees to 1.4×10^{-11} .

No parallelism within a tape. Chapter 9 gives the three reasons. Independent tapes parallelise; partitions do not.

The copy cost of `active` is undocumented in the header. It is described in this report and mitigated for independents, but a user who writes the obvious growing-container pattern still pays silently.

Two counters share a name. `Process::run_counter` (productive re-runs) and `TapeState::run_counter` (replay position) are different quantities with the same name inside what is now one object. Not a defect; a consolidation waiting to happen.

APICallState is still not wired in. An out-of-order free-function call is diagnosed at runtime rather than made impossible. The `Tape` surface makes most such orderings unrepresentable, which is the better answer, but the free functions remain public.